

SecureGate — Product Requirements Document (PRD)

Project Overview

SecureGate is a standalone authentication and security application built with Next.js 14, TypeScript, Prisma, PostgreSQL, NextAuth, Resend, and Vercel.

The purpose of SecureGate is to demonstrate production-grade authentication engineering, secure identity management, defensive programming, and proper full-stack architecture.

This is not a full SaaS product. It is a focused authentication system built to showcase:

- authentication correctness
- secure password handling
- token lifecycle management
- protected route handling
- security hardening
- production engineering practices

The application must be built progressively phase-by-phase with a strong focus on reliability, modularity, and security.

What SecureGate Will Build

Authentication Features

User Registration

- User can create an account using:
 - name
 - email
 - password
- All fields must be validated using Zod
- Passwords must be hashed using bcryptjs with salt rounds of 12
- Duplicate emails must be prevented
- Password strength indicator required

User Login

- User can log in using:
 - email
 - password
- Authentication handled using NextAuth Credentials Provider
- Sessions handled securely
- Error messages must not reveal whether:
 - email exists
 - password is wrong

Email Verification

- After registration:
 - generate verification token
 - save token in database

- send verification email using Resend
- Token expires after 15 minutes
- User clicks verification link
- Account becomes verified
- Token deleted after use

Protected Dashboard

- Dashboard accessible only to:
 - authenticated users
 - verified users
- Middleware must protect routes
- Unauthorized access redirects safely

Forgot Password

- User submits email address
- Generate password reset token
- Send password reset email
- Token expires after 1 hour
- User can reset password securely

Logout

- Destroy session securely
- Redirect user to login page

Rate Limiting

Protect:

- login endpoint
- forgot-password endpoint

Requirements:

- maximum 5 attempts
- per IP
- within 10 minutes

Security Hardening

- Hash passwords securely
- Expire all tokens
- Sanitize inputs
- Generic authentication errors
- Secure middleware redirects
- HTTP security headers

- Defensive programming practices
- No secrets hardcoded

Tech Stack & Setup

Layer	Tool	Purpose
---	---	---
Framework	Next.js 14 App Router	Full-stack React framework
Language	TypeScript	Type safety
Styling	Tailwind CSS	UI styling
Database	PostgreSQL	Persistent storage
ORM	Prisma ORM	Database access
Authentication	NextAuth.js	Session handling
Password Hashing	bcryptjs	Secure password hashing
Validation	Zod	Input validation
Email Service	Resend	Verification & reset emails
Email Templates	React Email	Transactional email UI
Rate Limiting	Upstash Redis	Brute-force protection
Deployment	Vercel	Production deployment
Version Control	GitHub	Source control

Environment Variables

Create ``.env.local`` at the root of the project.

Never commit ``.env.local``.

Required variables:

```
``env
```

```
DATABASE_URL=
```

```
NEXTAUTH_SECRET=
```

```
NEXTAUTH_URL=
```

```
RESEND_API_KEY=
```

```
RESEND_FROM=
```

```
UPSTASH_REDIS_REST_URL=
```

```
UPSTASH_REDIS_REST_TOKEN=
```

```
```
```

Create:

```
- `.env.example`
```

```
- `.gitignore`
```

Ensure:

```
- `.env.local` is ignored
```

```
- no secrets are exposed
```

```
- Vercel environment variables are configured
```

```

```

## # Build Phases

### # PHASE 1 — Scaffold & Database Schema

#### ## Goals

- Bootstrap Next.js 14 project
- Configure TypeScript
- Setup Tailwind CSS
- Initialize Prisma
- Connect PostgreSQL
- Create database schema

#### ## Database Models

##### ### User

- id
- name
- email
- password
- emailVerified
- createdAt

##### ### VerificationToken

- identifier
- token

- expires

### ### PasswordResetToken

- email
- token
- expires

## ## Requirements

- Run Prisma migrations
- Confirm database tables exist
- Create reusable Prisma client
- Push scaffold to GitHub before feature development

---

## # PHASE 2 — Authentication Core

### ## Goals

- Configure NextAuth Credentials Provider
- Implement secure login and registration
- Create protected dashboard

### ## Requirements

- bcrypt password hashing
- bcrypt compare during login



- JWT session strategy
- Zod validation
- Secure auth error messages
- Protected routes
- Accessible forms
- Loading states
- Password visibility toggle
- Password strength indicator

### ## Pages

- /register
- /login
- /dashboard

---

## # PHASE 3 — Email Verification Flow

### ## Goals

- Verify user emails securely

### ## Requirements

- Generate secure random tokens
- Store token in database
- Token expires after 15 minutes
- Send email using Resend

- Build verification route
- Handle:
  - expired token
  - invalid token
  - resend verification

## ## Pages

- /verify-email/[token]

---

## # PHASE 4 — Forgot Password Flow

### ## Goals

- Allow secure password reset

### ## Requirements

- Generate reset token
- Token expires after 1 hour
- Send password reset email
- Hash new password before saving
- Delete used token
- Never reveal whether email exists

## ## Pages

- /forgot-password
- /reset-password/[token]

---

## # PHASE 5 — Rate Limiting & Security Hardening

### ## Goals

- Protect application from abuse
- Harden authentication system

### ## Requirements

- Rate limit login endpoint
- Rate limit forgot-password endpoint
- Maximum:
  - 5 attempts
  - per IP
  - within 10 minutes

### ## Add Security Headers

- X-Frame-Options
- X-Content-Type-Options
- Referrer-Policy

### ## Security Requirements

- No stack traces exposed
- No account enumeration
- Secure middleware protection
- Environment variable safety
- Defensive redirects

---

## # PHASE 6 — UI Polish & Deployment

### ## Goals

- Improve user experience
- Prepare production deployment

### ## Requirements

- Responsive UI
- Accessible forms
- Validation messages
- Loading states
- Empty states
- Success states
- Error states
- Production-ready design

### ## Deployment

- Deploy to Vercel

- Configure environment variables
- Verify build works end-to-end

---

## # Architecture Requirements

### ## Folder Structure

```txt

src/

├─ app/

├─ components/

├─ lib/

├─ schemas/

├─ emails/

├─ hooks/

├─ types/

└─ middleware.ts

```

### ## Utilities

Create reusable utilities:

- prisma.ts
- auth.ts
- tokens.ts
- mail.ts

- rate-limit.ts
- validations.ts
- utils.ts

---

## # Security Requirements

The application must:

- never store plaintext passwords
- never trust client input
- validate all input on the server
- hash passwords using bcryptjs
- expire verification/reset tokens
- protect authenticated routes
- avoid leaking sensitive auth information
- use generic auth errors
- sanitize user input
- use defensive programming practices

---

## # Reflection.md Requirement

Create a `REFLECTION.md` file in the root of the repository.

This file is a required engineering analysis document.

The reflection document must include:

## ## Part 1 — What I Built

Short explanation of SecureGate and implemented features.

## ## Part 2 — What Surprised Me

One engineering challenge encountered and lessons learned.

## ## Part 3 — Engineering Analysis

Reference actual SecureGate code decisions and implementation details.

## ## Part 4 — One Thing I Would Refactor

Describe technical debt and proposed improvement.

## ## Part 5 — How This Changes How I Build

Reflection on authentication engineering, security, and product thinking.

IMPORTANT:

- Reflection must reference real implementation details
- Reflection must connect engineering decisions to SecureGate architecture
- Reflection must be written in plain English
- Reflection must align with actual codebase structure

---

## # Success Criteria

SecureGate is successful if:

- authentication works end-to-end
- passwords are securely hashed
- email verification works
- password reset works
- protected routes work correctly
- rate limiting blocks abuse
- tokens expire properly
- middleware redirects safely
- environment variables are secure
- deployment works on Vercel
- codebase is modular and production-ready
- reflection document aligns with implementation

---

## # Engineering Philosophy

SecureGate must be built with defensive engineering principles.

Assume:

- users make mistakes
- attackers probe weaknesses



- tokens expire
- sessions break
- requests fail
- cookies disappear
- inputs are malicious

The system must remain secure, predictable, and resilient under failure conditions.